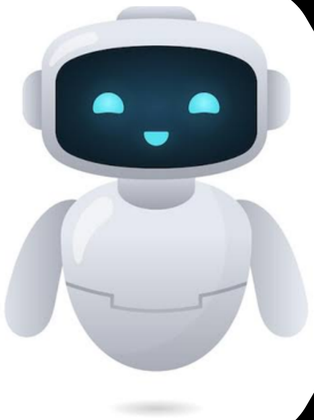
IIT GOA



ROS 2

BASICS TO ADVANCE

LECTURE-1



Presented by **AAYUSH VATS**

SYLLABUS

- **SETUP & ROS2 THEORY**
  → Docker and it’s setup.
  → ROS2 installation and it’s source.
  → Install TurtleSim
  → About nodes, topics, parameters, actions & launch nodes
  → RQT and it’s usage.

- **ROS2 PACKAGES AND COMMUNICATION METHODS**
  → Setting up colcon and basics of a ros2 package.
  → Types of communication.
  → Creating a cpp & python package.
  → Creating a publisher node.
  → Creating a subscriber node.

- **MORE ON CREATING NODES**
  → Create a service node.
  → Create a client node.
  → Custom srvs and msgs.
  → Parameters and creating a parameter node.
  → Creating custom cpp plugins.

SYLLABUS

- **ACTION CLIENT, BROADCASTERS-LISTNERS, TRANSFORMATIONS**
  → Creating action-client & server nodes.
  → Launch files & creating them.
  → Integrating a launch file into a package.
  → Transformations.
  → Creating a broadcasting node.
  → Creating a listener node.

- **GAZEBO & URDF**
  → Installing Gazebo & RVIZ 2.
  → Creating a URDF file for a simple mobile robot.
  → SDF & the process to convert URDF into SDF.
  → ROS2-Gazebo bridge.
  → ROS1-ROS2 bridge.

- **RVIZ 2 & ROS DEBUGGING METHODS**
  → Rviz 2 usage.
  → ROSBAG2 install & usage.
  → Common debugging methods.
  → ROS2 communication with multiple devices basic commands, tips & tricks.

SYLLABUS

- **BONUS**
  → ROS2-control.
  → Basics on Nav2 package.
  → MicroROS.

PREREQUISITES:

CPP – https://www.learncpp.com/

PYTHON – https://www.geeksforgeeks.org/python/python-programming-language-tutorial/

DOCKER

- An open source project that automates the deployement of software applications inside **CONTAINERS** by providing an additional layer of abstraction and automation of OS-level virtualization on linux.
- Today most industries use VMs to run software applications. Virtual Machines (VMs) run applications inside a guest OPERATING SYSTEM, which runs on a virtual hardware powered by server's host OS.
- Containers offer a logical packaging mechanism in which applications can be abstracted from the environment in which they actually run. This decoupling allows container-based applications to be deployed easily and consistently, regardless of whether the target environment is a private data center, the public cloud, or even a developer's personal laptop. This gives developers the ability to create predictable environments that are isolated from the rest of the applications and can be run anywhere.
- **SETUP** on your laptop is straightforward. Download **DOCKER Desktop** on windows, linux, Mac. Follow the instructions from the following link:

  https://docs.docker.com/desktop/setup/install/windows-install/
- Once installed open your docker terminals and try the following command:

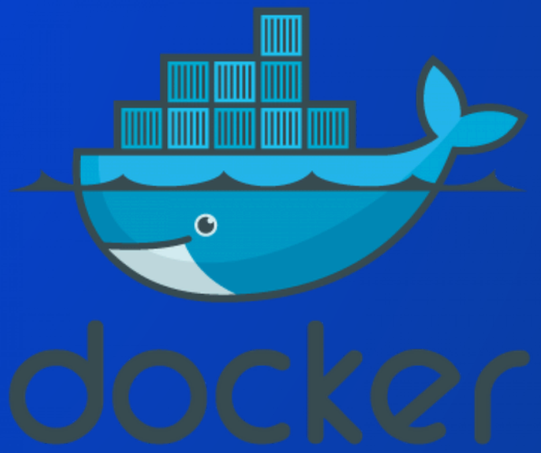
  $ docker run hello-world

  # if you get the following response:

  Hello from Docker!

  This message shows that your installation appears to be working correctly.

  ...



DOCKER

- Some important DOCKER commands:

1.) **docker run “container name”**. (This command runs the container if it doesn’t work try adding “sudo” before writing the command)

2.) **docker ps** (shows the current running container).

3.) **docker run –it “container name”** (runs the container with an interactive interface. This is a very useful command for our purpose).

4.) **docker rm “container id”** (to avoid your disk space to be eaten up. always use this command before running a new container)

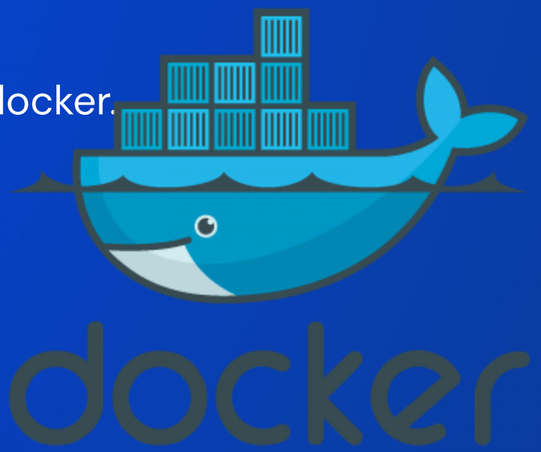
5.) **docker container prune** (WARNING! This will remove all stopped containers. Are you sure you want to continue? [y/N] y)

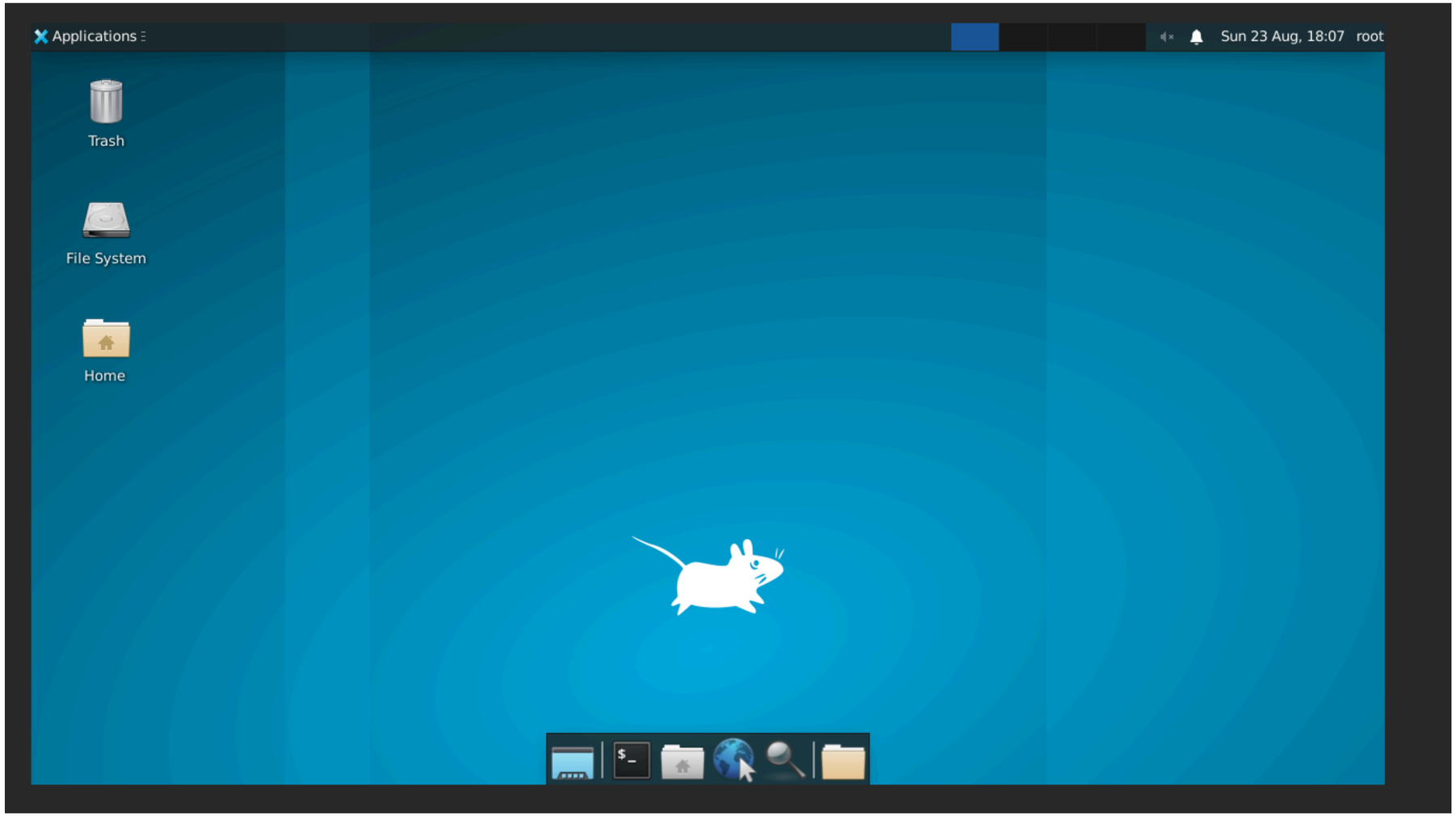
6.) **docker pull “image”** (you can download an already available image)

- After verifying your docker is installed properly visit the link.

https://hub.docker.com/r/aayush11101998/ubuntu22-novnc

- Do read the description and step properly to start a virtual image of ubuntu 22.04 using your docker.





YOUR CONTAINER IMAGE IN NOVNC

WHAT IS ROS ?



ROS is an open-source, meta-operating system, for robots.

It provides services that you expect from an Operating system

- Hardware abstraction.
- Low-level device control.
- Message-passing between processes.
- Package management.
- Implementation of commonly-used functionalities

Requirements for running ROS into your systems

- OS: Unix-based platforms. Primarily Ubuntu.
- Windows 11 in the recent years, but not recommended.

Components of ROS

- Nodes
- Topics
- Services
- Packages
- Launch Files

DIFFERENCE BETWEEN ROS1 & ROS2

FEATURE	ROS1	ROS2
MIDDLEWARE	CUSTOM ROS COMMUNICATION SYSTEM	DDS
MASTER	REQUIRES ROS MASTER	NO CENTRAL MASTER REQUIRED
REAL-TIME SUPPORT	LIMITED	BETTER REAL-TIME SUPPORT
SECURITY	LIMITED	DDS-BASED SECURITY
QoS	LIMITED	EXTENSIVE
CURRENT DEPLOYEMENT	LEGACY/ END OF LIFE	ACTIVE SUPPORT ECOSYSTEM

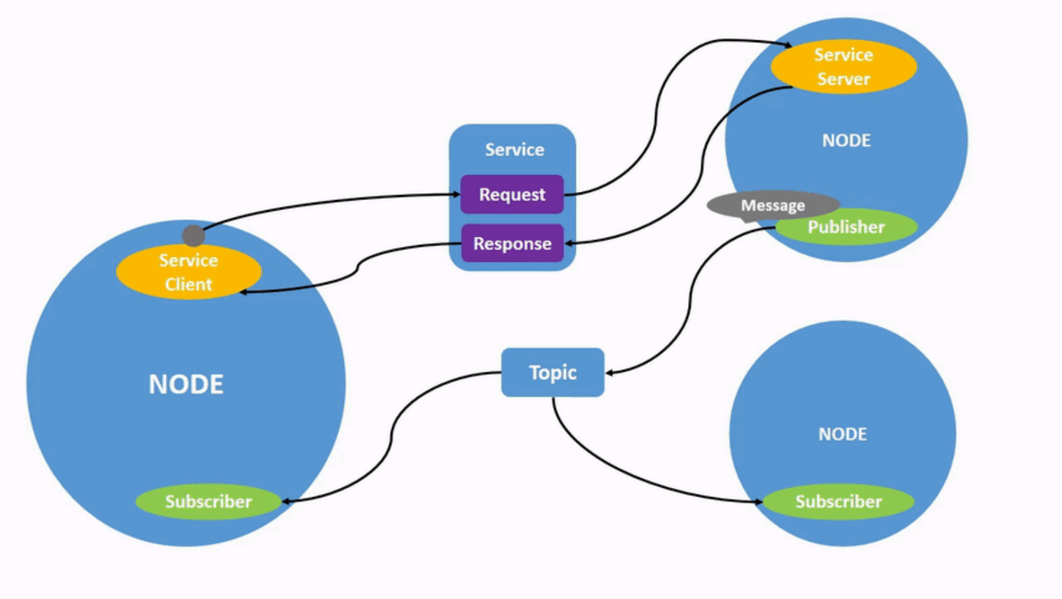
INSTALLING ROS2



- Visit the link mentioned here:
  https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html
- Using the information on the webpage you can easily install ROS2 humble into your newly made docker container.
- You have to install **ros-humble-desktop** for the current course.
- Once installed open your .bashrc file through the terminal via nano or gedit editor.
- At the very bottom paste the command:
  source /opt/ros/humble/setup.bash
- *The above command ensures that your ROS2 has been sourced into the OS and is now ready to use.*

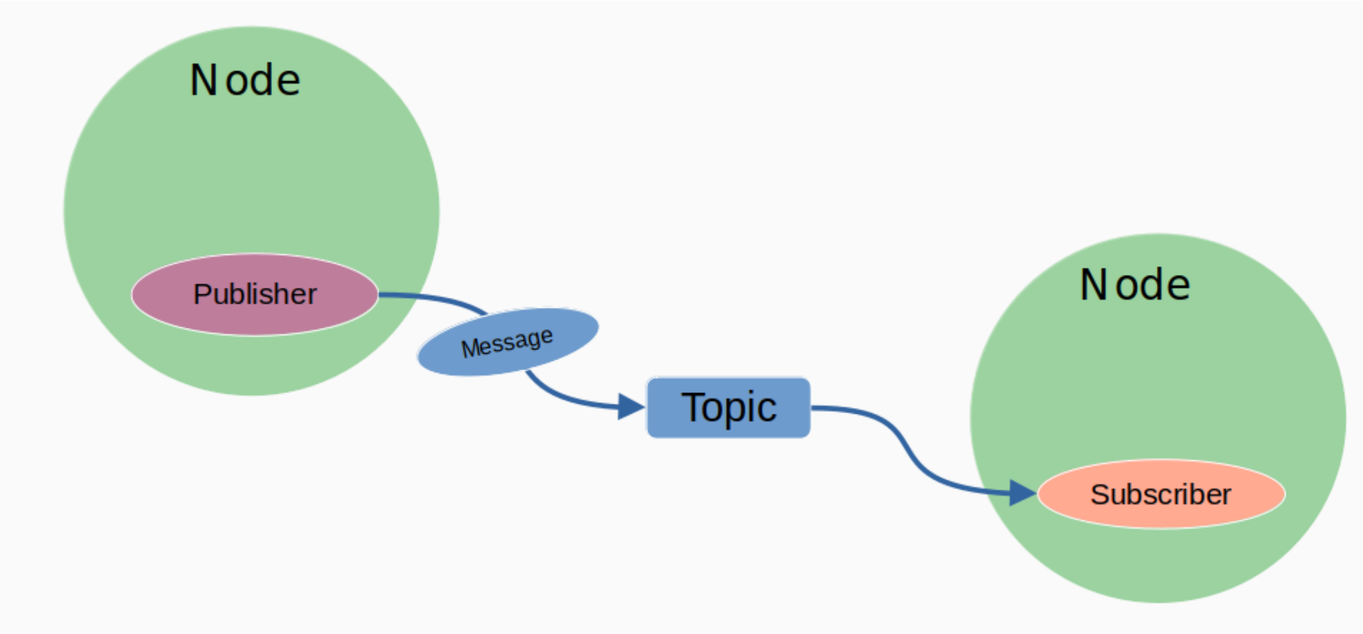
ROS-NODES

- Nodes act as the processes.
- Packages in ROS are modular by nature and also they should be practiced in the similar fashion for easiness.
- Each node is the singular part of the modular package.
- They communicate with each other via topics, services, actions, parameters.



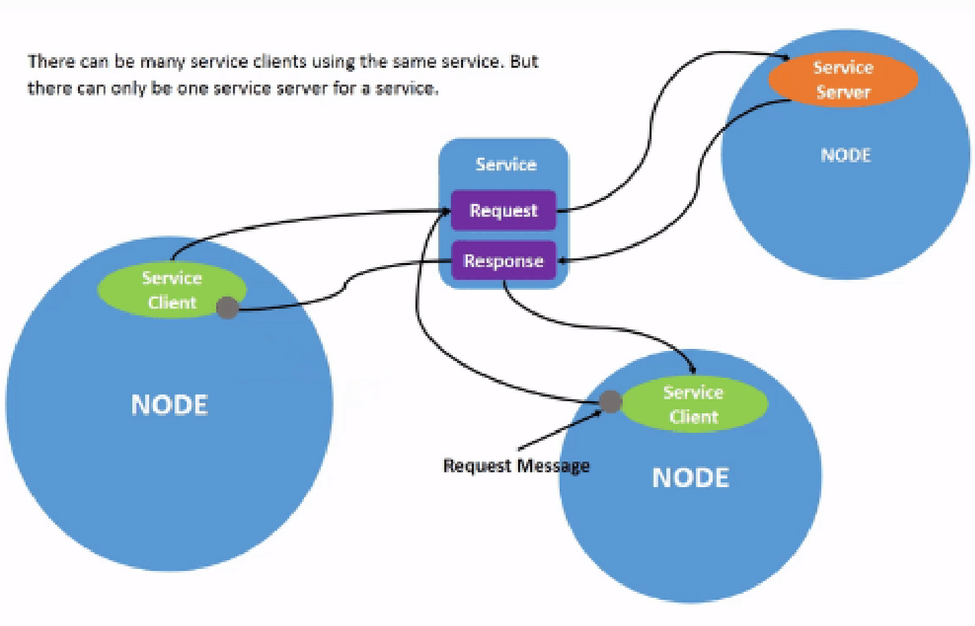
ROS-TOPICS

- ROS breaks complex system into nodes and the topics are a vital element that acts as bus for nodes to exchange messages.
- A node or many nodes simultaneously may use topics to publish data and also to subscribe data from it.



ROS-SERVICES

- Just like topics, services are another mode using which nodes communicate.
- Unlike publishing or subscribing it is more of a call and response mechanism.
- Topics allows a continuous flow of data to whichever node subscribing to them. Services provides data only when some node requests for it.

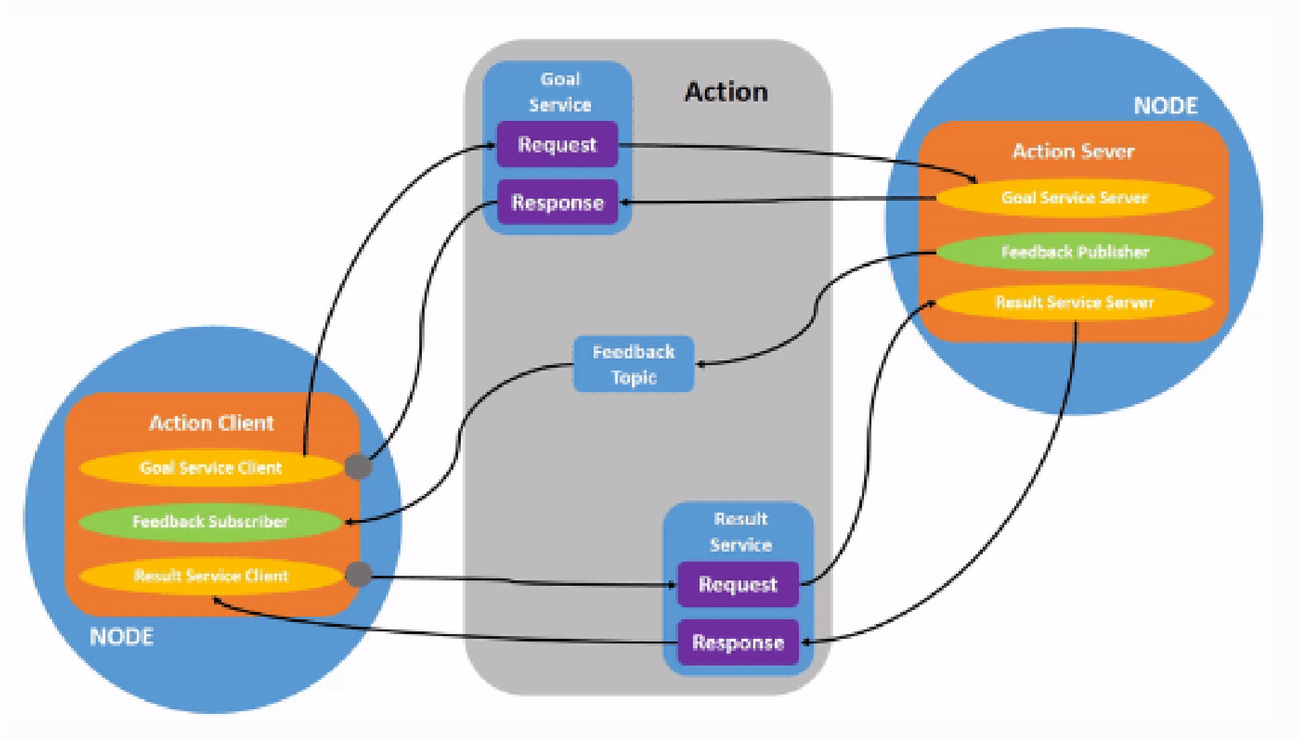


ROS-PARAMETERS

- A parameter is a configuration value of node.
- In ROS2 each node maintains it’s own node settings called parameters.
- Parameters can be addressed by node names, node namespaces.
- Providing a parameter to node is optional.
- Each parameter consists of a key, value and a descriptor. The key is a string, the value is one of the mentioned types: bool, int64, float64, string, byte[], bool[], float 64[], string64[].By default the descriptors are empty, but can contai parameter descriptions, value range etc.

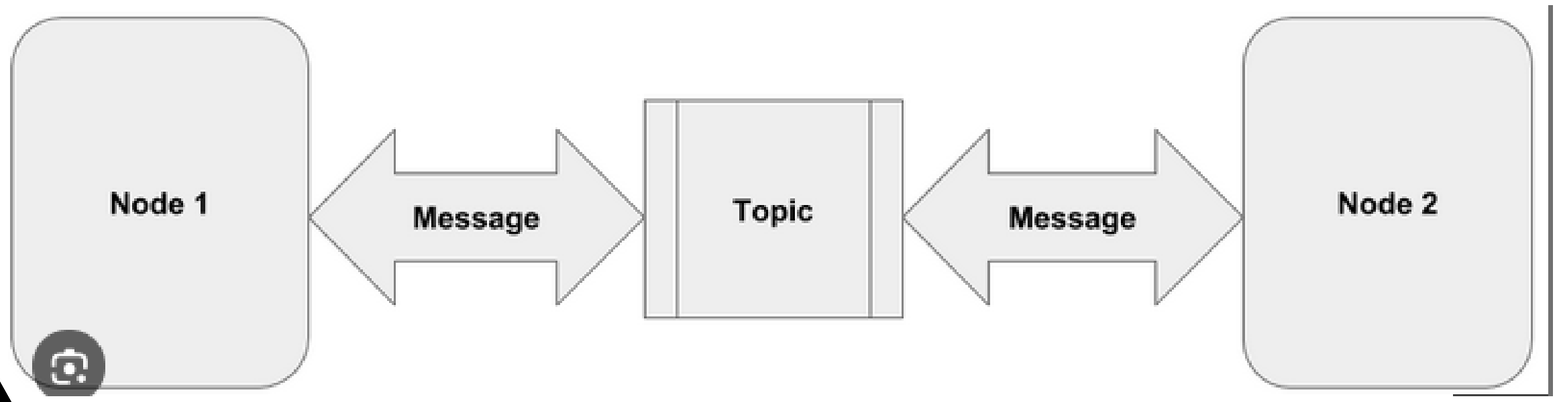
ROS-ACTIONS

- Actions are one of the communication types in ROS2 intended for long processes.
- These can be divided into three parts: GOAL, FEEDBACK, RESULT.
- They are very similar to services except for the fact that they can be canceled. Also, they provide feedback.



ROS-MESSAGES

- Another way of communication between nodes is via messages.
- A message is a simple data structure comprising typed fields.
- A message usually gets converted from one data type to another between nodes, for communication.



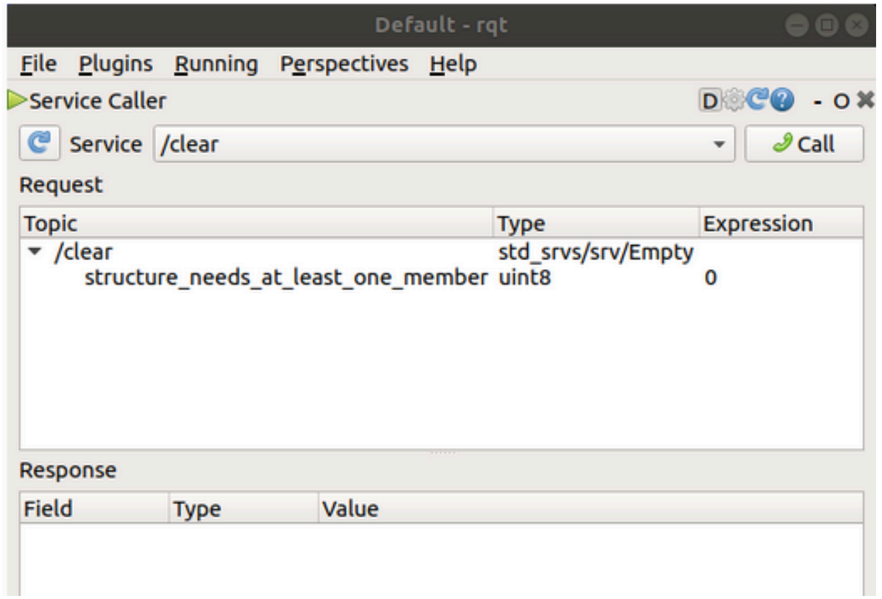


TURTLESIM & RQT

sudo apt update
sudo apt install ros-humble-turtlesim

- Some common commands:
  → ros2 pkg excecutables turtlesim
  → ros2 run turtlesim turtlesim_node
  → ros2 run turtlesim turtle_teleop_key

sudo apt install ros-humble-rqt-common-plugins



UNDERSTANDING RQT AND RQT_CONSOLE

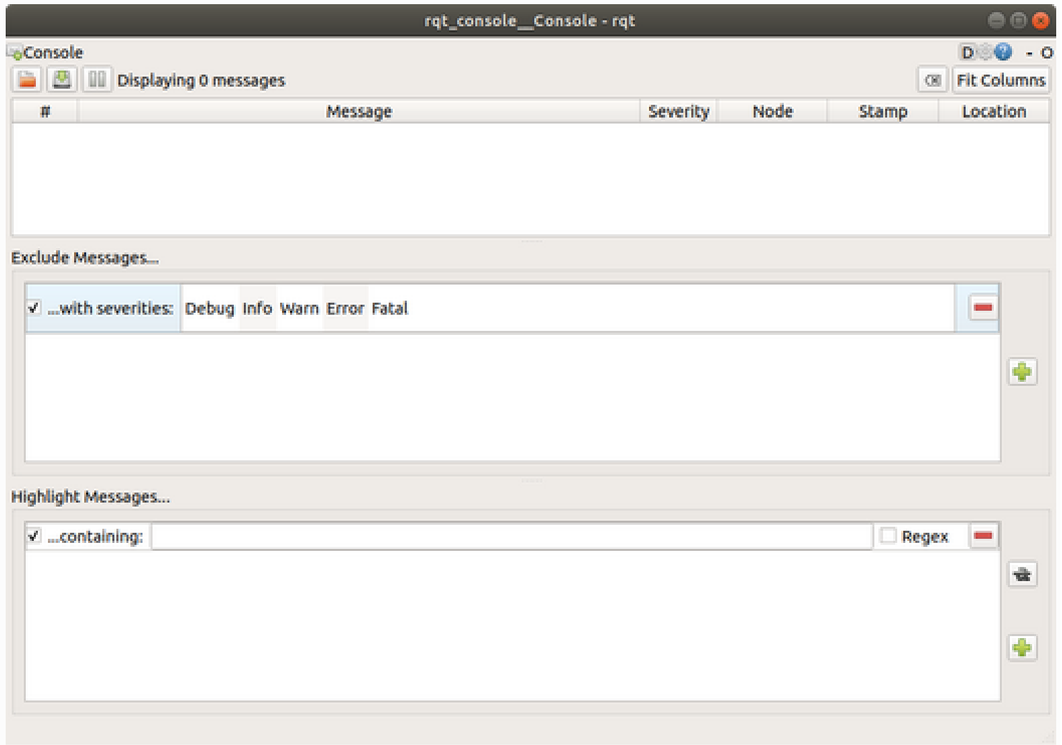
To view logs

Background

It is a GUI_tool used to introspect log_messages in ROS 2. It is a very useful tool when debugging, as you can collect log_messages and then view them closely in an organized manner.

ros2 run rqt_console rqt_console

is the command to open and run rqt_console.



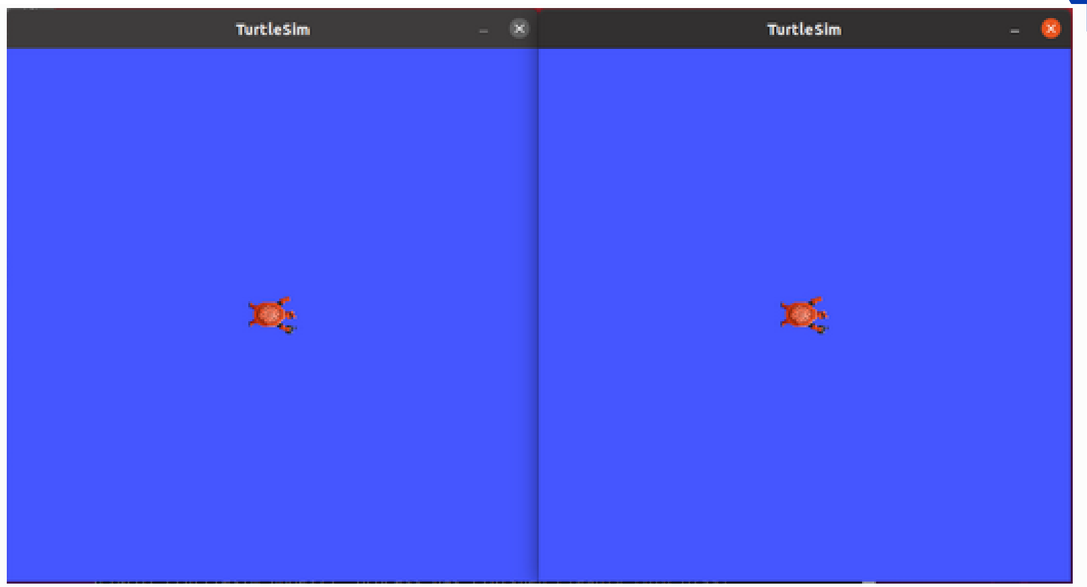
LAUNCH NODES

- As mentioned in the previous slides ROS 2 packages are modular in nature.
- The more the modular structure the more complex it will be to select and run the nodes made to run a set of tasks.
- Launch nodes provide a more systematic and convinient way to run multiple nodes using just a single terminal command.
- These nodes also ensure all the node configurations run at once.

- Try running "**ros2 launch turtlesim multisim.launch.py**"

```
from launch import LaunchDescription
import launch_ros.actions


def generate_launch_description():
    return LaunchDescription([
        launch_ros.actions.Node(
            namespace='turtlesim1', package='turtlesim',
            executable='turtlesim_node', output='screen'),
        launch_ros.actions.Node(
            namespace='turtlesim2', package='turtlesim',
            executable='turtlesim_node', output='screen'),
    ])
```



THANK YOU

FOR YOUR TIME AND ATTENTION TODAY. WE APPRECIATE YOUR ENGAGEMENT.



9812930249

aayush25263101
@iitgoa.ac.in

LAB-4, ACEDAMIC BLOCK-1